Python Automata Library (PAL) Cheat Sheet

Conventions
import PythonAutomataLibrary as pal · Safe mode is default. Call **pal.FastMode**() before constructing any PAL object. Put substantive model kernels under **@pal.njit(cache=True)**. dims gives grid size per axis: (nx,) for 1D, (nx, ny) for 2D, (nx, ny, nz) for 3D. Negative dimensions enable wrapping; out-of-range coordinates wrap on wrapped axes and are skipped on nonwrapped axes. x... means x, x,y, or x,y,z in 1D/2D/3D.

Create state

pal.NewGrid(dims, dtype): Create one value per site; supports bool, fixed-width integer, and float dtypes.
pal.NewAgentGrid(dims, numAgentProps=0, isStackable=False): Create mobile agents with per-agent properties. isStackable permits multiple agents per site; dims=() makes a nonspatial collection.
pal.NewPopGrid(dims, capacity=None): Store integer population counts per site; capacity limits total population.
pal.NewPDEgrid(dims): Create a continuous field with queued synchronous updates and diffusion/advection operators.
pal.NewIList(): Create a mutable integer list for collecting/reusing query results.
pal.NewMultinomial(): Create a sampler for binomial/multinomial population draws.

Shared lattice geometry

Properties: **xDim** / **yDim** / **zDim**: axis sizes; **nDims**: number of axes; **wrapX** / **wrapY** / **wrapZ**: wrapping flags.
ToI(x...): Convert coordinates to a linear site index; inverse methods are ItoX/Y/Z(i).
grid.Hood(hood, x...): Apply relative coordinate offsets, e.g. ((x1,y1),(x2,y2),...), at x.... Special-case hoods: **pal.MooreHood**(*), **pal.VonNeumannHood**(*), **pal.CircleHood**(*,rad,*). * represents shared arguments: dim (number of dimensions) and excludeCenter=False (omit the center when true).

Grid / common indexing

Indexing: **grid[i]** reads/writes by linear index; **grid[x...]** uses dimensional coordinates.
Slices: Return detached NumPy copies, not views; write back explicitly to change the grid.
Box(x1,x2,...): Select a rectangular region: (xLow,xHigh,yLow,yHigh,...), with exclusive upper bounds.

Draw / output

pal.StartPixWindow(...): Create a pixel buffer and display window; headless=True supports noninteractive rendering.
pix[x,y] = RGB: Set a pixel color in the shared pixel buffer.
Color utilities: **pal.RGB**(r,g,b) packs red, green, and blue channels into a color; **pal.GetR**(color), **pal.GetG**(color), and **pal.GetB**(color) extract channels; **pal.ColorScale**(...) maps values to colors.
PixWindow controls: **win.Update**() refreshes the display; **win.Save**(path,block=True) saves an image; **win.Close**() releases the window.
PixWindow GIF: **win.StartGif**(path,delay=100) starts recording; **win.AddGifFrame**(block=False) appends a frame; **win.StopGif**() finishes.
pal.StartOpenGLWindow(xDim,yDim,zDim=None,width=800,height=800,...): Create the renderer and return (draw, win): draw collects primitives; win handles display, updates, and output. Omit zDim for 2D; set it for 3D. headless=True supports noninteractive rendering.
draw.Circle(rad,color,x,y,z=None): Add a circle or sphere-like primitive with the given radius and color.
draw.Box(xLen,color,x,y,z=None,yLen=None,zLen=None): Add a box centered at a position; specify axis lengths to control its size.
draw.BoxSQ(color,x,y,z=None): Add a box aligned to a lattice site.
draw.Line(width,color,x1,y1,x2,y2,z1=None,z2=None): Add a line segment between two 2D/3D points.
draw.Borders(width,color): Draw grid borders with the requested line width and color.
win.Camera(x,y,z,yaw=None,pitch=None): Set the camera position and optional yaw/pitch angles for a 3D scene.
OpenGL frame: **win.Background**(color) sets the background; **draw.Clear**() clears collected primitives; **win.Clear**() clears the scene; **win.Update**() renders.
OpenGL output: **win.IsOpen**() checks the window; **win.Save**(path,block=False) saves an image; **win.Close**() releases resources.
OpenGL GIF: **win.StartGif**(...) begins recording; **win.AddGifFrame**(...) adds a frame; **win.StopGif**(...) finishes.

Lists + randomness

IList methods: **Append**(i) adds an integer; **Clear**() empties the list; **Random**() picks an entry; **Shuffle**() reorders entries; **All**() copies the list; **Iter**() traverses without copying.
RNG methods: **pal.Seed**(seed) seeds PAL's RNG; **pal.Random**() draws uniform [0,1); **pal.RandInt**(n) draws an integer in [0,n). PAL's RNG is separate from NumPy's.
Multinomial methods: **m.Binomial**(n,p) draws a binomial count; **m.Setup**(...) configures multinomial probabilities; **m.Sample**(...) draws counts.

AgentGrid

NewAgentSQ(x...) / **MoveSQ**(a,x...): Create/move agents by integer site coordinates; linear-site variants are also available.
NewAgent(x...) / **Move**(a,x...): Create/move agents by continuous coordinates (off-lattice).
grid[a,p]: Read or write property p for agent a.
Agent position: **I**(a) gives the linear site index; XSQ/YSQ/ZSQ(a) give lattice coordinates; X/Y/Z(a) give continuous coordinates.
Alive(a) / **Dispose**(a): Check whether an agent is still valid, or remove it from the grid.
GetPop(): Count agents currently in the grid.
AgentsAt(x...) / **LastAgent**(x...): Query all agents at a site or just its last agent; useful for stackable grids.
AgentsInRadius(...): Find nearby agents, respecting periodic boundaries.
counts[x...]: Read site occupancy counts without modifying them.
Iteration: **agents.All**() returns a detached snapshot, safe for iteration while adding or disposing agents.
DispWrapX/Y/Z(p1,p2): Compute the shortest displacement along a periodic axis.

PopGrid + PDEgrid

Synchronous updates: **Add**(v,...) queues a change; **Update**() applies queued changes together; **Reset**() clears current and pending values. Indexed assignment changes current state immediately.
pop.GetPop(): Return total population across all sites.
pop.All(): Return occupied site indices (not their counts).
PDE setup: **field.SetTimeSpaceStep**(dt, dx[,dy,dz]) sets time and axis spacing; explicit updates require stable time steps.
Diffusion methods: **Diffusion**(...) uses the standard scheme; **DiffusionMask**(...) applies a mask; **DiffusionField**(...) uses spatially varying coefficients; **DiffusionInterfaces**(...) specifies interfaces; **DiffusionADI**(...) uses an implicit scheme; DiffusionRadialCircle/Sphere(...) handles radial geometry.
Advection methods: **Advection**(...) uses constant velocity; **AdvectionField**(...) uses spatially varying velocity;

Python Automata Library (PAL) Cheat Sheet

AdvectionInterfaces(...) specifies interface velocities.

More detail: Manual · API Guide · API Reference (Documentation/).